

API Güvenliği Neden Hayati Önem Taşır?

NTP II – Hafta 12



Giriş: API Güvenliği Neden Hayati Önem Taşır?

Bu hafta, modern web API'lerinin güvenliğini sağlamanın kritik önemi üzerine odaklanacağız. Özellikle siber güvenlik verileri (IoC gibi hassas bilgiler) söz konusu olduğunda, API erişiminin kontrol edilmesi bir zorunluluktur [1].

API güvenliğinin ihlali, veri kaybına, itibar zedelenmesine ve yasal sonuçlara yol açabilir. Kaynakta belirtildiği gibi, yetkisiz erişime açık endpoint'ler ('GET /indicators') büyük bir güvenlik fiyaskosudur [2].

Güvenlik İhlali Senaryosu: Acil Durum!

Karşılaştığımız senaryo: API'deki tüm IoC (Indicator of Compromise) verileri internete tamamen açık [2].

Hatta yetkisiz herkes, 'DELETE /indicators/{id}' endpoint'ini kullanarak verileri silebilir [2].

Bu durum, API'nin kimlik doğrulama mekanizmalarının bulunmadığı anlamına gelir. Bu güvenlik açığı, anında çözüm gerektiren en üst düzey öncelikli bir sorundur.



Çözüm Adımları: Güvenliğin Yeniden İnşası

Bu fiyaskoyu gidermek için üç ana görevimiz var:

1. Kayıt Mekanizması: Kullanıcıların (Analistlerin) e-posta ve parola ile kayıt olabileceği '/user/signup' endpoint'i oluşturulmalı ve parolalar güvenli bir şekilde hash'lenmeli [2].
2. Giriş Mekanizması: Kayıtlı kullanıcıların 'erişim token'i' alabileceği '/login' veya '/token' endpoint'i oluşturulmalı [3].
3. Endpoint Koruma: GET, POST, PUT, DELETE dahil olmak üzere tüm '/indicators' endpoint'leri, sadece geçerli bir token gönderen kullanıcılar tarafından erişilebilir olmalıdır [3].



Modül Öğrenme Hedefleri

Bu modülü tamamladığinizda şunları anlamış olacaksınız [1]:

1. API güvenliğinin neden kritik olduğunu.
2. OAuth2 'Password Flow' standardını.
3. 'Bearer Token' kavramını.
4. JSON Web Tokens (JWT) yapısını (Header, Payload, Signature).
5. Güvenlik odaklı OOP: Kullanıcı modelinin veritabanına eklenmesini.
6. Parola güvenliği: Passlib ve bcrypt kullanarak hashing işlemlerini.

Hashing (Karma) vs. Şifreleme (Encryption)

Bu, bilişim güvenliği öğrencileri için en önemli ayrımı temsil eder [3].

****Şifreleme (Encryption):**** Veriyi gizlemek ve daha sonra geri getirmek (iki yönlü) için kullanılır (örn: AES, RSA). Veri, bir anahtar ile şifrelenir ve aynı veya farklı bir anahtar ile çözülür.

****Hashing (Karma):**** Tek yönlü bir süreçtir. Veriden benzersiz, sabit uzunlukta bir dize (hash) oluşturur (örn: bcrypt, SHA-256). Orijinal veri (parola) asla hash'ten geri çıkarılamaz.

Kural: Parolalar asla şifrelenmez, daima hash'lenir [3].



Hashing Algoritması Seçimi: Neden bcrypt?

Eski ve hızlı algoritmalar (SHA-1, MD5) günümüzde parola saklamak için tehlikelidir, çünkü GPGPU'lar ve Rainbow Tabloları ile kolayca kırılabilirler.

****bcrypt:**** Yavaş olacak şekilde tasarlanmıştır. Bu 'yavaşlık', brute-force saldırılarına karşı direnç sağlar. Hesaplama maliyeti (cost factor) ayarlanabilir, bu da donanım geliştikçe güvenliğini artırmayı sağlar.

Kaynakta belirtildiği gibi, FastAPI uygulamasında bcrypt kullanılarak passlib kütüphanesi ile karma işlemi uygulanacaktır [1].



Parola Hashleme Uygulaması: Passlib Kütüphanesi

Passlib, Python'da parola karma işlevlerini soyutlayan kapsamlı bir kütüphanedir.

****Kullanım Amacı:**** Parolaları hash'lemek ('get_password_hash' fonksiyonu) ve kullanıcının girdiği parolayı veritabanındaki hash ile karşılaştırmak ('verify_password' fonksiyonu) için kullanılır [4].

****Kurulum:**** Passlib'in bcrypt desteğiyle kurulması gerekir: 'pip install python-passlib[bcrypt]' [5].



Güvenlikte Nesne Yönelimli Programlama (OOP)

Kullanıcı verilerini veritabanında güvenli bir şekilde saklamak için bir kullanıcı modeli ('User') tanımlanmalıdır [1].

****models.py içeriği:****

```
'class User(Base):'  
'    __tablename__ = users'  
'    id = Column(Integer, primary_key=True)'  
'    email = Column(String, unique=True)'  
'    hashed_password = Column(String)'
```

****Önemli Not:**** 'hashed_password' alanı, düz metin parolayı DEĞİL, sadece karma edilmiş parolayı saklar [5].

Veri Şemaları (Schemas) ve Pydantic

FastAPI, veri doğrulama ve serileştirme için Pydantic modellerini kullanır. Güvenlik akışımız için gerekli şemalar 'schemas.py' dosyasında tanımlanmalıdır [5].

****Gereken Şemalar:****

1. 'UserCreate': Kayıt sırasında kullanıcıdan alınan e-posta ve parola.
2. 'UserInDB': Veritabanında saklanan (hashlenmiş parola dahil) kullanıcı bilgileri.
3. 'Token': Başarılı girişte döndürülecek erişim token'ı ve tipi.
4. 'TokenData': JWT'nin içerdiği veriyi temsil eder (örn: kullanıcının e-postası) [5].

OAuth2 Standardı ve Rolü

OAuth2 (Open Authorization 2.0), yetkilendirme için kullanılan bir çerçevedir. Bu, kullanıcıların bir hizmete (örneğin API'mize) üçüncü taraf uygulamalar üzerinden güvenli erişim vermesini sağlar.

Bu senaryoda, biz sadece standart bir parçası olan **'Password Flow'**u kullanıyoruz [1].

Password Flow, uygulamanın doğrudan kullanıcının kullanıcı adı ve parolasıyla kimlik doğrulama yapıp erişim token'i aldığı, eski bir yöntem olmasına rağmen bazı güvenilir birinci taraf uygulamalarda hala kullanılabilir.



OAuth2 Password Flow Akışı

Bu akış, kullanıcının doğrudan sunucuya kimlik bilgilerini sağlamasına dayanır:

1. Kullanıcı, istemciden (tarayıcı veya uygulama) e-posta/kullanıcı adı ve parolayı girer.
2. İstemci bu bilgileri '/login' (veya '/token') endpoint'ine POST eder.
3. Sunucu kimlik bilgilerini doğrular.
4. Başarılı olursa, sunucu bir 'Erişim Token'i' döndürür [3].
5. Bu token, sonraki tüm korumalı isteklerde kullanılır.

Bearer Token Kavramı

Başarılı kimlik doğrulamasından sonra kullanıcıya geri dönen token, genellikle bir **'Bearer Token'**dir [1, 6].

'Bearer' (Taşıyıcı), token'a sahip olan kişinin (taşıyıcının) bu token'i kullanarak kaynağa erişim yetkisine sahip olduğunu belirtir.

Bu token, korumalı endpoint'lere yapılan isteklerde HTTP başlığı olarak gönderilir: 'Authorization: Bearer <token_dizesi>' [7].

JSON Web Tokens (JWT) Tanımı

JWT, iki taraf arasında güvenli bir şekilde bilgi alışverişi yapmak için kullanılan kompakt, URL güvenli bir yapıdır.

JWT'ler genellikle, kullanıcının kimliğini doğrulamak ve yetkilendirme bilgilerini taşımak için kullanılır.

Bir JWT, üç ana parçadan oluşur: Header, Payload ve Signature [1].

JWT Yapısı: Header (Başlık)

Header, token'ın nasıl oluşturulduğu hakkında meta bilgileri içerir.

İki temel alan barındırır:

1. ****alg (Algorithm):**** Token'ı imzalamak için kullanılan kriptografik algoritma (örn: HS256, RS256) [4].
2. ****typ (Type):**** Token'ın türü (genellikle 'JWT').

Bu kısım, Base64 URL Safe formatında kodlanır ve token'ın ilk parçasıdır.

JWT Yapısı: Payload (Yük)

Payload, iletilmek istenen veriyi veya ****iddiaları (claims)**** içerir. Claims, kullanıcının kimliği, yetkileri ve token'ın geçerliliği gibi bilgileri ifade eder.

Önemli Claims örnekleri:

- * ****sub (Subject):**** Token'ın ait olduğu kişi veya varlık (genellikle kullanıcının e-postası) [7].
- * ****exp (Expiration Time):**** Token'ın geçerliliğini yitireceği zaman (Unix zaman damgası) [7].
- * ****iss (Issuer):**** Token'ı düzenleyen taraf (API sunucusu).

Bu kısım, Base64 URL Safe formatında kodlanır ve token'ın ikinci parçasıdır.

JWT Payload Güvenlik Uyarısı

Çok önemli bir güvenlik kuralı: JWT'nin Payload kısmı base64 formatında kodlanır, ancak ****ŞİFLENMEZ**** [8].

Bu, herkesin token'i alıp kolayca Base64 decode ederek içeriğini (sub, exp vb.) görebileceği anlamına gelir.

****Asla Payload içine gizli veriler (parolalar, kritik kişisel bilgiler) konulmamalıdır!**** Sadece kimlik doğrulama için gerekli minimum veriler (sub ve exp) konulur [8].



JWT Yapısı: Signature (İmza)

Signature, JWT'nin en kritik güvenlik bileşenidir. **JWT'nin bir 'şifreleme' değil, bir 'imza' olduğunu vurgulayın** [8].

Signature, şu bileşenler kullanılarak oluşturulur:

1. Kodlanmış Header.
2. Kodlanmış Payload.
3. Sunucunun yalnızca kendisinin bildiği gizli bir anahtar ('SECRET_KEY') [4, 7].
4. Header'da belirtilen Algoritma (örn: HMAC SHA256).

Bu imza, Payload'un iletim sırasında değiştirilmediğini garanti eder.

JWT Doğrulama Mekanizması

Bir kullanıcı, korumalı bir kaynağa erişmek için bir JWT gönderdiğinde (Authorization: Bearer başlığında), sunucu doğrulama yapar [7]:

1. Sunucu, kendi gizli anahtarını kullanarak token'in Header ve Payload'undan ****aynı imzayı yeniden hesaplar****.
2. Yeniden hesaplanan imza, token'daki mevcut imza ile eşleşmelidir.
3. Eşleşirse, token'in içeriğinin (Payload) değiştirilmediği anlaşılır.
4. Daha sonra sunucu, Payload içindeki 'exp' süresinin dolup dolmadığını ve 'sub' alanının geçerli olup olmadığını kontrol eder [7].

Kodlama Süreci: Adım 1 – Bağımlılıklar

Projenin güvenlik ihtiyaçları için iki ana bağımlılığa ihtiyacımız var [5]:

1. ****Parola işlemleri için:**** passlib ve bcrypt.
'pip install python-passlib[bcrypt]'
2. ****JWT işlemleri için:**** python-jose (JSON Object Signing and Encryption) veya PyJWT.
'pip install python-jose[cryptography]' (tercih edilen) [5].

Bu adımlar, ders akışında ilk 15 dakikalık bölümü oluşturur [5].

Veritabanı tablosunu tanımlayan 'User' modeli, 'models.py' dosyasına eklenir. Bu model, SQLAlchemy gibi bir ORM (Object-Relational Mapping) aracıyla veritabanı etkileşimini sağlar [5].

Ana sütunlar:

- * 'id': Birincil anahtar.
- * 'email': Benzersiz olmalı ve indexlenmeli (giriş için kullanılacak) [5].
- * 'hashed_password': Parolanın karma edilmiş halini saklar.

Kodlama Süreci: Adım 3 – Şemalar (Schemas)

Giriş ve çıkış verilerinin Pydantic ile standardize edilmesi gerekir [5].

****Token Şeması:**** Başarılı girişte istemciye döndürülecek yapıyı tanımlar [6].

```
'class Token(BaseModel):'  
'    access_token: str'  
'    token_type: str =    bearer'
```

****TokenData Şeması:**** JWT payload'undaki bilgileri (sub) temsil eder.

Tüm güvenlik, hashing, token oluşturma ve doğrulama mantığı, ana uygulamadan ayrı, modüler bir 'auth.py' dosyasına taşınmalıdır [4].

Bu, kodun temizliğini, test edilebilirliğini ve yeniden kullanılabilirliğini artırır. Bu adım, ders akışının en uzun kısmını (90 dakika) oluşturur [4].



auth.py: Konfigürasyon ve Gizli Anahtar

Kimlik doğrulama için gerekli sabitler 'auth.py' içinde tanımlanmalı veya daha iyisi, bir '.env' dosyasından okunmalıdır [4].

****Gerekli Sabitler:****

- * 'SECRET_KEY': JWT'leri imzalamak için kullanılan, kesinlikle gizli tutulması gereken anahtar [4].
- * 'ALGORITHM': Token imzalama algoritması (örn: HS256) [4].
- * 'ACCESS_TOKEN_EXPIRE_MINUTES': Token'in geçerlilik süresi (ömrü).

auth.py: OAuth2PasswordBearer Nesnesi

FastAPI'nin OAuth2 desteğini etkinleştirmek için bir 'OAuth2PasswordBearer' nesnesi oluşturulmalıdır [2].

Bu nesne, istemcilerin token'ı nereden alacaklarını bilmesini sağlar.

```
'oauth2_scheme = OAuth2PasswordBearer(tokenUrl='token' [4]
```

'tokenUrl= token' parametresi, token'ı almak için kullanılan endpoint'in '/token' olduğunu belirtir [4].

Passlib kullanarak iki temel yardımcı fonksiyon tanımlanır [4]:

1. `get_password_hash(password)`: Düz metin parolayı alır ve bcrypt kullanarak hash'lenmiş dizesini döndürür. Bu, `/users/signup` sırasında kullanılır.
2. `verify_password(plain_password, hashed_password)`: Kullanıcının girdiği parolayı, veritabanında saklanan hash ile karşılaştırır. Bu, `/token` (login) sırasında kullanılır [4].

Bu fonksiyon, '/token' endpoint'inde başarılı kimlik doğrulamasından sonra çağrılır [4].

****işleyiş:****

1. Payload verisi (örn. '{sub: user@email.com}') alınır [7].
2. JWT'nin son kullanma süresi ('exp') hesaplanır [7].
3. Veri, gizli anahtar ('SECRET_KEY') ve algoritma kullanılarak imzalanır.
4. Uzun JWT dizesi (Header.Payload.Signature) döndürülür [7].

auth.py: Kullanıcı Doğrulama (authenticate_user)

Bu fonksiyon, giriş isteği geldiğinde kimlik bilgilerini kontrol eder [4]:

1. Veritabanından, sağlanan e-posta (username) ile eşleşen kullanıcıyı arar.
2. Kullanıcı bulunamazsa, doğrulama başarısızdır.
3. Kullanıcı bulunursa, 'verify_password' fonksiyonu çağrılarak düz metin parola, saklanan hash ile karşılaştırılır [4].
4. Başarılı olursa, 'User' objesi döndürülür. Başarısız olursa, 'None' döndürülür.

auth.py: JWT'yi Çözme (decode_jwt)

Koruma altına alınmış endpoint'lere yapılan isteklerde gelen JWT'nin çözülmesi gerekir. Bu, token'ın geçerli olup olmadığını ve içindeki kimlik bilgisini (sub) kontrol etmeyi içerir.

Eğer imza geçerliyse ve süresi dolmadıysa, payload'daki veriler ('TokenData' şemasına uygun olarak) alınır.



auth.py: Mevcut Kullanıcıyı Getiren Dependency

FastAPI'nin en güçlü özelliği olan 'Depends' mekanizmasını kullanarak, korumalı her endpoint'ten önce çalışacak bir 'dependency' (bağımlılık) oluşturulur [4, 9].

****işlevi:****

1. 'OAuth2PasswordBearer' nesnesini kullanarak HTTP başlığından token'i otomatik olarak alır.
2. Token'i çözer ve içindeki 'sub' (e-posta) bilgisini alır.
3. Bu e-posta ile veritabanında kullanıcıyı bulur.
4. Eğer token geçersizse, süresi dolduysa veya kullanıcı bulunamazsa, 401 Unauthorized (Yetkisiz) hatası fırlatılır [6].
5. Başarılı olursa, giriş yapmış 'models.User' objesini döndürür [9].

main.py: Uygulama Başlangıcı

Ana uygulama dosyamız 'main.py', gerekli kütüphaneleri ve kendi modüllerimizi içe aktarmalıdır [10]:

```
'from fastapi import Depends, FastAPI, HTTPException'  
'from fastapi.security import OAuth2PasswordRequestForm'  
'from . import auth, schemas, models'  
'app = FastAPI()'
```

Ek olarak, veritabanı oturumu ('get_db') gibi bağımlılıklar da mevcut olmalıdır [6, 10].

Endpoint Oluşturma: Kullanıcı Kayıt (/users/signup)

Bu endpoint, kullanıcıların güvenli bir şekilde API'ye kayıt olmasını sağlar [2, 10].

****Metot:**** HTTP POST

****Yol:**** '/users/signup'

****İş Akışı:****

1. Gelen 'user: schemas.UserCreate' verisi alınır.
2. Parola, 'auth.get_password_hash' fonksiyonu kullanılarak karma haline getirilir.
3. Yeni kullanıcı, 'hashed_password' ile veritabanına eklenir [10].
4. Kullanıcıya başarılı kayıt mesajı döndürülür.

Endpoint Oluşturma: Giriş ve Token Alma (/token)

Bu endpoint, kullanıcının erişim token'ını aldığı yerdir [3, 6].

****Metot:**** HTTP POST

****Yol:**** '/token'

****Gereken Bağımlılık:**** 'OAuth2PasswordRequestForm = Depends()' [6]. Bu, FastAPI'nin OAuth2 standardına uygun form verisini (username ve password) otomatik olarak ayrıştırmasını sağlar.

****İş Akışı:****

1. 'auth.authenticate_user' çağrılır.
2. Başarısız kimlik doğrulaması durumunda (kullanıcı bulunamazsa veya parola yanlışsa), 401 yetkisiz HTTPException fırlatılır [6].

Token Başarılı Yanıtı

Kimlik doğrulama başarılı olduğunda, sunucu 'auth.create_access_token' fonksiyonunu kullanarak bir JWT oluşturur [6].

Sunucu, istemciye aşağıdaki formatta bir yanıt döndürür [6]:

```
{access_token: access_token_dizesi, token_type: bearer}
```

İstemci, bu token'i alır ve korumalı kaynaklara erişmek için kullanmaya başlar.

Endpoint Koruma Mekanizması

Tüm kritik IoC endpoint'leri (GET, POST, PUT, DELETE /indicators) artık korunmalıdır [3].

Koruma, endpoint fonksiyonunun parametrelerine eklenen özel bir bağımlılık ('Depends') ile sağlanır [6, 9].

```
'current_user: models.User = Depends(auth.get_current_user)' [9]
```

Bu, endpoint'e ulaşılmadan önce 'auth.get_current_user' fonksiyonunun çağrılmasını ve token'in kontrol edilmesini zorunlu kılar.

Korumalı Endpoint Örneği: GET /indicators

IoC verilerini okuma endpoint'i artık güvenlidir [2, 6]:

```
'@app.get(/indicators, response_model=List)'  
'def read_indicators(db: Session = Depends(get_db), current_user:  
models.User = Depends(auth.get_current_user))'
```

****Güvenlik Garantisii:**** Fonksiyonun kod kısmı ('indicators = db.query...') sadece, token'i geçerli olan ve kimliği 'current_user' objesi ile belirlenen kullanıcılar için yürütülür [9].

Eğer istek, korumalı bir endpoint'e ulaşırsa, bu demektir ki:

1. Kullanıcının bir token'ı vardı.
2. Token, 'Authorization: Bearer' başlığıyla gönderildi [7].
3. Sunucu, token'ın imzasını kendi gizli anahtarıyla başarıyla doğruladı [7, 9].
4. Token'ın süresi (exp) dolmamıştı.
5. 'current_user' değişkeni, API'nin verileri kimin adına çektiğini bilir [9].

Bu, artık tüm CRUD işlemlerinin (POST, PUT, DELETE) de aynı şekilde korunabileceği anlamına gelir [9].



JWT Süre Yönetimi (Expiration)

Erişim token'larının sonsuza kadar geçerli olması büyük bir güvenlik riskidir. Eğer bir token çalınırsa, saldırgan sonsuza kadar erişime sahip olur.

Bu yüzden her JWT, bir son kullanma tarihi ('exp' claim) içermelidir [7].

Sunucu, token'ı doğrularken 'exp' süresinin dolup dolmadığını kontrol eder. Süresi dolan token'lar otomatik olarak reddedilir, bu da saldırı penceresini sınırlar.



Token Yenileme Stratejileri (Refresh Tokens)

Kısa ömürlü erişim token'ları güvenliği artırsa da, kullanıcı deneyimini bozabilir.

****Refresh Token (Yenileme Token'i):**** Daha uzun ömürlü, genellikle tek kullanımlık bir token'dır. Erişim token'ının süresi dolduğunda, istemci refresh token'i kullanarak sunucudan yeni bir erişim token'i ister.

Bu, kullanıcının sık sık yeniden giriş yapmasını engellerken, güvenlik riskini minimumda tutar. (Kaynaklarda doğrudan bahsedilmese de, akademik bir sunumda önemli bir konsepttir.)



Salt Kullanımı ve Brute-Force Koruması

Passlib ve bcrypt gibi modern hashing algoritmaları, her parola için benzersiz bir 'salt' (tuz) otomatik olarak ekler. Salt, hash'e eklenen rastgele bir dizedir.

****Amacı:****

1. Aynı parolaya sahip iki kullanıcının farklı hash değerlerine sahip olmasını sağlar.
2. Saldırganların önceden hesaplanmış hash tabloları (Rainbow Tabloları) kullanmasını imkansız hale getirir.

Salt, hash ile birlikte veritabanında saklanır.

JWT Standartları: JWS ve JWE

JWT'ler aslında daha geniş bir standart ailesine aittir:

1. ****JWS (JSON Web Signature):**** Kaynak metinde ele aldığımız yapı. Token'in içeriğinin değiştirilmediğini garanti etmek için sadece imzalanır (gizlenmez) [8].
2. ****JWE (JSON Web Encryption):**** Token'in hem imzalandığı hem de şifrelendiği yapıdır. Hassas bilgileri taşımak gerektiğinde JWE kullanılır, ancak JWT'nin yaygın kullanımı JWS'dir.

Role-Based Access Control (RBAC)

Kimlik doğrulama (Authentication) sadece 'kim olduğunu' belirler. Yetkilendirme (Authorization) ise 'ne yapabileceğini' belirler.

JWT'yi kullanarak yetkilendirme ekleyebiliriz.

****Uygulama:**** Payload'a kullanıcının rollerini ('roles: ['admin', 'analyst']') ekleyebiliriz.

****FastAPI'de:**** 'get_current_user' dependency'si, kullanıcının rollerini kontrol edecek şekilde genişletilebilir. Örneğin, 'DELETE /indicators' endpoint'ine sadece 'admin' rolüne sahip kullanıcılar erişebilir.



Kullanıcı kayıt sürecinde hata yönetimi kritiktir:

1. ****E-posta Benzersizliği:**** Veritabanına yeni bir kullanıcı eklenirken, e-postanın zaten mevcut olup olmadığı kontrol edilmelidir [5]. Eğer mevcutsa, uygun bir 400 Bad Request hatası döndürülmelidir.
2. ****Veri Doğrulama:**** Pydantic ('schemas.UserCreate') sayesinde e-posta formatının geçerliliği otomatik olarak kontrol edilir.

Login Sürecinde Fırsat Penceresi (Timing Attack)

Eğer parola doğrulama süreci ('verify_password') çok hızlı ve tutarsız bir şekilde çalışırsa, saldırganlar zamanlama farklarını kullanarak doğru parolayı tahmin etmeye çalışabilirler.

****Çözüm:**** bcrypt gibi algoritmalar yavaşlatılarak ve 'verify_password' fonksiyonları, doğrulamanın başarısız olup olmadığına bakılmaksızın yaklaşık aynı sürede çalışacak şekilde tasarlanarak zamanlama saldırıları önlenir.

FastAPI Güvenlik Mekanizmaları Özeti

FastAPI, kimlik doğrulamasını soyutlamak için Python'ın bağımlılık enjeksiyon sistemini kullanır:

1. 'Depends(get_db)': Veritabanı oturumu gibi kaynakların yönetimi.
2. 'Depends(OAuth2PasswordRequestForm)': Login sırasında form verisinin standartlaştırılması [6].
3. 'Depends(auth.get_current_user)': Token doğrulama, JWT çözümüleme ve kullanıcı nesnesini endpoint'e taşıma [9].

Bu mimari, ana endpoint kodunun temiz kalmasını ve sadece iş mantığına odaklanmasını sağlar.



Swagger UI ve Otomatik Dokümantasyon

FastAPI'nin en büyük avantajlarından biri, yazdığımız endpoint'lerin otomatik olarak interaktif bir dokümantasyon (Swagger UI) oluşturmaktır.

****Güvenlik Entegrasyonu:**** 'OAuth2PasswordBearer' nesnesini tanımladığımızda, Swagger UI'da otomatik olarak bir ****'Authorize'** (Kilit) düğmesi** görünür [9].

Öğrenciler, bu düğmeyi kullanarak '/token' endpoint'inden aldıkları token'i yapıştırıp korumalı endpoint'leri test edebilirler [9].



Token Testi: Nasıl Çalışır?

Test akışı:

1. ****Adım 1:**** '/token' endpoint'ine gidilir ve geçerli e-posta/parola ile POST isteği yapılır.
2. ****Adım 2:**** Dönüş olarak gelen 'access_token' kopyalanır.
3. ****Adım 3:**** Swagger UI'daki 'Authorize' düğmesine tıklanır.
4. ****Adım 4:**** Token, 'Bearer' önekiyle birlikte (veya sadece token dizesi olarak) yapıştırılır.
5. ****Adım 5:**** Koruma altındaki '/indicators' endpoint'ine GET isteği yapılır. Eğer token geçerliyse, veri döner; geçersizse 401 hatası alınır.

Güvenlik uygulamalarında doğru HTTP hata kodlarının döndürülmesi kritiktir.

1. ****401 Unauthorized:**** Kimlik doğrulama başarısız olduğunda (token yok, token geçersiz, yanlış parola) kullanılır [6]. Bu, kullanıcının kimlik bilgilerini sağlaması gerektiğini gösterir.
2. ****403 Forbidden:**** Kimlik doğrulama başarılı, ancak kullanıcının kaynağa erişim yetkisi yok (yetkilendirme hatası – RBAC senaryolarında).

JWT'nin en büyük avantajlarından biri, durum bilgisiz (stateless) olmasıdır. Sunucunun token'i doğrulamak için veritabanına bakmasına gerek yoktur (sadece imzayı kontrol etmesi yeterlidir).

Bu, mikroservis mimarilerinde önemlidir: Bir kullanıcının kimlik doğrulamasını yapan 'Auth Service', bir JWT oluşturur. Diğer 'Indicator Service' ise sadece bu JWT'nin imzasını kontrol ederek kullanıcıyı doğrulayabilir (Veritabanı erişimi gerekmez).



Özet: Kritik Güvenlik Öğretileri

1. Hassas veriler içeren tüm API endpoint'leri mutlaka korunmalıdır [2, 3].
2. Parolalar asla düz metin veya şifrelenmiş olarak saklanmamalıdır; daima ****bcrypt**** ile hash'lenmelidir [1, 3].
3. JWT sadece bir ****imza**** mekanizmasıdır, Şifreleme değildir; bu yüzden Payload'a gizli veri koymayın [8].
4. OAuth2 'Bearer Token' standardı, istemciden sunucuya yetki taşımak için kullanılır [7].
5. FastAPI'deki 'Depends' mekanizması, kodu temiz tutarken güçlü güvenlik uygulamaları yazmayı sağlar [4, 9].



Ek Okuma: Çalınan Token'lar ve İptal Listeleri

JWT'lerin durum bilgisiz olması bir avantajdır, ancak token çalındığında veya kullanıcı çıkış yaptığında (logout) bir dezavantaj yaratır.

Sunucu, token'ın süresi dolana kadar (exp) token'ı geçerli kabul eder.

****Çözüm:**** Kullanıcı çıkış yaptığında, token'ı bir 'İptal Listesi' (Revocation List) veya Kara Liste'ye (Blacklist) eklemek. Sunucu, her istekte önce bu listeyi kontrol eder. Bu, durum bilgisiz mimariye durum bilgisi eklemek demektir.



CSRF Koruması (Cross-Site Request Forgery)

Bearer token'ları genellikle tarayıcıda 'localStorage' içinde saklanır. Ancak, CSRF saldırıları ve XSS (Cross-Site Scripting) saldırıları bu token'ları çalabilir.

****En İyi Uygulama:**** Eğer token'lar 'HttpOnly' ve 'SameSite=Strict' çerezleri içinde saklanırsa, XSS ve CSRF riskleri azalır. OAuth2 ve JWT genellikle SPA'lar (Single Page Application) için tasarlandığından, bu bir tasarım kararıdır.



İmzalama Algoritmaları Karşılaştırması

JWT imzalama için iki ana kategori vardır:

1. ****Simetrik (HS256):**** Token'ı imzalamak ve doğrulamak için aynı gizli anahtar ('SECRET_KEY') kullanılır [4, 7]. En yaygın ve basittir.
2. ****Asimetrik (RS256):**** İmzalama için özel anahtar, doğrulama için genel anahtar kullanılır. Bu, mikroservis mimarilerinde önemlidir, zira tüm servislerin gizli anahtarı bilmesi gerekmez, sadece genel anahtarla doğrulamaları yeterlidir.

Güvenlik fonksiyonlarımızı barındıran 'auth.py' dosyasındaki başlangıç konfigürasyonunu inceleyelim. Gizli anahtarın kod içine gömülmemesi esastır.

İdeal olarak, 'SECRET_KEY' çevre değişkenlerinden (örneğin '.env' dosyasından) yüklenmelidir [4].

Bu, üretim ortamında anahtarın kolayca değiştirilebilmesini ve kaynak kodun ifşa olması durumunda anahtarın gizli kalmasını sağlar.



Parola doğrulama fonksiyonunun temel işleyişi:

```
'def authenticate_user(db, username, password):'  
' user = get_user(db, username=username)'  
' if not user:'  
'     return False'  
' if not verify_password(password, user.hashed_password):'  
'     return False'  
' return user'
```

Bu süreç, kullanıcının mevcut olup olmadığına bakılmaksızın, her durumda, yetkisiz erişimi engellemek için hata fırlatmalıdır [6].

'create_access_token' fonksiyonu içinde 'datetime' kütüphanesi kullanılarak token'in geçerlilik süresi hesaplanır:

```
'expire = datetime.utcnow() +  
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)'
```

Payload içine yalnızca gerekli (non-sensitive) veriler eklenir, özellikle 'sub' (subject) ve 'exp' (expiration) [7].

'/users/signup' endpoint'i, HTTP POST isteğini kabul eder ve 'schemas.UserCreate' objesini bekler [10].

Bu endpoint'in yanıtı, genellikle parolanın hashlenmiş versiyonunu içermeyen bir 'schemas.User' modeli olmalıdır (güvenlik açısından).

Kaynak kodda, kullanıcı oluşturma ve parolayı hashleme işlemlerinin 'auth.py' içindeki fonksiyonlar kullanılarak yapıldığı belirtilmiştir [10].

'/token' endpoint'i, 'OAuth2PasswordRequestForm' bağımlılığını kullanarak form verilerini alır [6].

Başarısız giriş durumunda, bir 'HTTPException' fırlatılır:

```
'raise HTTPException(status_code=401, detail=Incorrect email or password)' [6]
```

Bu hata mesajı, saldırganlara e-posta mı yoksa parola mı yanlış olduğu bilgisini vermemek için kasıtlı olarak genel tutulmuştur.

IoC verilerini okuyan endpoint'in korunması [9]:

'current_user: models.User = Depends(auth.get_current_user)' bağımlılığı, tüm doğrulama işlemlerinin bu satırda tamamlanmasını zorlar.

Eğer doğrulama başarılı olursa, 'current_user' değişkenine, veritabanından çekilmiş oturum açmış kullanıcı nesnesi atanır. Bu nesne, daha sonra yetkilendirme (authorization) kontrolleri için kullanılabilir.

JWT'de Nonce ve Replay Saldırıları

Eğer bir JWT çalınırsa ve süresi dolmadıysa, saldırgan tarafından tekrar kullanılabilir (Replay Attack).

Nonce (Number used once) veya JTI (JWT ID) gibi benzersiz tanımlayıcılar Payload'a eklenebilir.

Sunucu, bu JTI'yi veritabanında saklayarak, aynı JTI'ye sahip bir token'in tekrar kullanılıp kullanılmadığını kontrol edebilir. Bu, JWT'ye durum bilgisi eklemek anlamına gelir, ancak güvenliği artırır.



API Gateway ve Güvenlik İlişkisi

Üniversite düzeyinde, modern API mimarilerinde kimlik doğrulama genellikle doğrudan FastAPI uygulamasında yapılmaz.

Bunun yerine, bir API Gateway (örn. Kong, Ocelot) tüm gelen istekleri yakalar.

Bu Gateway, token'i doğrular (JWT Signature kontrolü) ve yalnızca geçerli istekleri FastAPI uygulamasına yönlendirir. Bu, FastAPI'nin üzerindeki yükü azaltır ve güvenlik politikasını merkezileştirir.

Güvenlik Tasarım İlkesi: En Az Ayrıcalık

API güvenliğinde temel bir ilke: Her kullanıcının, görevini yerine getirmek için ihtiyacı olan en az ayrıcalığa sahip olması gerekir.

Eğer bir 'Analist' kullanıcısı sadece IoC verilerini okumak zorundaysa ('GET /indicators'), onun 'DELETE /indicators' yetkisi olmamalıdır.

Bu, JWT Payload'una rol bilgisini ekleyerek ve 'get_current_user' fonksiyonunda rol kontrolü yaparak uygulanır.



Veritabanı Güvenliği: Salted Hash Depolama

Bir saldırgan veritabanına erişim sağlasa bile, ****hashlenmiş parolalar**** ve bunlarla birlikte saklanan ****salt'lar**** sayesinde düz metin parolaları anında elde edemez [5].

Saldırganın yine de her bir hash'i tek tek brute-force veya dictionary saldırılarıyla kırması gerekir. Bcrypt'in kasıtlı yavaşlığı, bu saldırıları pratik olarak imkansız hale getirir.



Güvenli Bağlantı (HTTPS/TLS)

Token'lar (JWT) ve parolalar (giriş sırasında) düz metin olarak iletildiği anda (Payload şifreli değildir), bir man-in-the-middle (ortadaki adam) saldırısına açıktır [7].

Bu nedenle, API'ye yapılan tüm isteklerin mutlaka ****HTTPS/TLS**** üzerinden yapılması zorunludur. TLS, kanalın uçtan uca şifrelenmesini sağlayarak token'in ağ trafiği sırasında çalınmasını engeller.



Güvenlikte Denetim Kayıtları (Auditing)

Güvenlik olaylarının izlenmesi, herhangi bir siber güvenlik uygulamasının temelidir.

****Ne Kaydedilmeli?****

1. Başarılı/Başarısız '/login' denemeleri (IP adresi, zaman).
2. '/users/signup' ile yeni kullanıcı oluşturulması.
3. Kritik işlemlerin (DELETE /indicators gibi) kimin tarafından yapıldığı ('current_user' bilgisi kullanılarak).

Bu kayıtlar, bir ihlal durumunda olayın kök neden analizini yapmayı sağlar.

Rate Limiting (Hız Sınırlaması)

Brute-force saldırıları, otomatik botların '/login' veya '/users/signup' gibi endpoint'lere milyonlarca deneme yapmasıyla gerçekleşir.

****Rate Limiting Uygulaması:**** Bir IP adresinden belirli bir zaman aralığında (örn. 5 dakika içinde 5 deneme) izin verilen istek sayısını sınırlandırmak.

Bu, bcrypt'in sağladığı korumaya ek olarak, saldırganın deneme hızını fiziksel olarak sınırlar.



Kullanıcı Deneyimi ve Güvenlik Dengesi

JWT'lerin süresi çok kısaysa (örn. 5 dakika), kullanıcılar sürekli olarak yeniden kimlik doğrulaması yapmak zorunda kalır (kötü deneyim).

Süre çok uzunsa (örn. 1 hafta), token çalınması riski artar.

****Pratik Çözüm:**** Erişim token'ını kısa (15-30 dakika) tutmak ve yenileme token'ı (refresh token) ile kullanıcı deneyimini iyileştirmek [4, 7].

JWT Standartları: Claims Tipleri

JWT payload'undaki iddialar (claims) üç tiptir:

1. ****Registered Claims:**** JWT standardı tarafından tanımlanmışlardır (örn. 'iss', 'exp', 'sub'). Bunlar zorunlu değildir ama önerilir [7].
2. ****Public Claims:**** Çakışmaları önlemek için IANA JSON Web Token Registry'de tanımlanmış veya çarpışma önleyici bir isim uzayı olarak tanımlanmış iddialardır.
3. ****Private Claims:**** Gönderici ve alıcı arasında paylaşılan, özel tanımlanmış iddialardır (örn. 'user_role').

OAuth2 Password Flow Dezavantajları

Password Flow, istemcinin kullanıcının parolasını alması gerektiği için genellikle modern standartlarda tavsiye edilmez.

****Modern Alternatifler:****

* ****Authorization Code Flow with PKCE:**** En güvenli akıştır. Parola, istemciye değil, doğrudan kimlik sağlayıcıya (IdP) verilir.

Bu modülde (Kaynak 1), ilk olarak en temel OAuth2 entegrasyonu gösterilmiş olsa da, daha güvenli akışlar akademik olarak bilinmelidir.



Veri Şemaları: TokenData Detayı

Token çözüldükten sonra elde edilen veri, Pydantic modeli 'schemas.TokenData'ya dönüştürülür:

```
'class TokenData(BaseModel):'  
  ' username: Optional[str] = None'
```

Bu, JWT Payload'undaki 'sub' alanını tutar. 'auth.get_current_user' bu bilgiyi kullanarak veritabanından kullanıcıyı çeker.

FastAPI'de Ortak Bağımlılıklar

'auth.get_current_user' fonksiyonunun kendisi, başka bir bağımlılığa (veritabanı oturumu - 'get_db') ihtiyaç duyar [4, 9].

Bu, FastAPI'de bağımlılıkların zincirleme (nested) olarak çalışabildiği anlamına gelir. 'get_current_user' çağrıldığında, otomatik olarak 'get_db'yi de çağırır.

Gizli anahtarların ('SECRET_KEY') '.env' dosyalarında saklanması gerekir [4].

Bu dosyalar:

1. ****Sürüm Kontrolünden Hariç Tutulmalı:**** '.gitignore' dosyasına eklenmelidir.
2. ****Üretim Ortamında:**** Güvenli bir anahtar yönetimi sisteminde (AWS Secrets Manager, Azure Key Vault) saklanmalıdır.



Güvenliğin, geliştirme sürecinin en başına ('sola kaydırılması') taşınması gerekir.

Bu modülde gördüğümüz gibi, kullanıcı modeli ('models.py') ve giriş/kayıt endpoint'leri projenin en başında, veri okuma/yazma endpoint'lerinden önce tasarlanmalıdır [2].

Güvenlik, sonradan eklenen bir özellik değil, mimarinin temel taşı olmalıdır.

JWT'de Kid (Key ID) Kullanımı

Uygulama büyüdükçe, gizli anahtarı periyodik olarak değiştirmek (rotasyon) gerekir. Eğer anahtar değişirse, eski token'lar anında geçersiz olur.

****Çözüm:**** JWT Header'ına bir 'kid' (Key ID) eklemek.

Bu, sunucunun token'ı çözerken hangi gizli anahtarın kullanılması gerektiğini bilmesini sağlar. Bu sayede, aynı anda birden fazla geçerli anahtarı destekleyebiliriz.



Saldırı Tespiti ve Önleme Sistemleri (IDS/IPS)

Yazılım düzeyindeki kimlik doğrulama, API'yi korumanın sadece bir parçasıdır.

IDS (Intrusion Detection System) ve IPS (Intrusion Prevention System) gibi ağ düzeyinde araçlar, anormal trafik desenlerini (örn. birden fazla başarısız giriş denemesi) tespit ederek API'ye ulaşmadan saldırıları engelleyebilir.



Token Doğrulama Hızının Etkisi

JWT doğrulaması (imza kontrolü) oldukça hızlıdır, çünkü veritabanı sorgusu gerektirmez.

Ancak, 'get_current_user' dependency'si, JWT'den gelen e-posta ile veritabanında kullanıcı nesnesini (rol, ID vb. için) arar. Bu bir veritabanı sorgusudur.

****Performans Optimizasyonu:**** Eğer sadece yetkilendirme için JWT kullanılıyorsa, veritabanı sorgusu atlanabilir, ancak bu, veritabanında kullanıcının aktif olup olmadığını kontrol etme yeteneğini ortadan kaldırır.



PyJWT ve Python-JOSE Karşılaştırması

Kaynakta, bağımlılıklar için 'python-jose' veya 'PyJWT' seçenekleri sunulmuştur [5].

****Python-JOSE:**** Genellikle JWS, JWE ve JWA (JSON Web Algorithms) standartlarını destekleyen daha kapsamlı bir kütüphanedir.

****PyJWT:**** Daha sade, sadece JWT standardına odaklanmıştır. FastAPI projelerinde her ikisi de kullanılabilir, ancak 'python-jose' daha geniş kriptografik seçenekler sunar.



Hata Yönetimi Detayı: 401 Yanıt Yapısı

FastAPI'nin 'HTTPException(status_code=401, detail=Incorrect email or password)' ile döndürdüğü yanıt, istemciye JSON formatında standart bir hata mesajı sunar [6].

****Önemli:**** Hata mesajlarının, saldırganlara sistem hakkında bilgi sızdırmayacak kadar genel olması güvenlik açısından bir zorunluluktur (örn. 'Email is incorrect' yerine 'Incorrect email or password').



Token Kullanımı: Çerezler mi, Başlıklar mı?

Erişim token'ının istemcide nerede saklanacağı önemlidir:

1. ****HTTP Başlığı (Authorization: Bearer):**** Kaynakta belirtilen yöntem budur [7]. XSS saldırılarına karşı daha savunmasızdır, ancak kullanımı kolaydır ve SPA'lar için standarttır.
2. ****HttpOnly Çerezler:**** XSS saldırılarına karşı daha güvenlidirler, çünkü JavaScript çereze erişemez. Ancak CSRF koruması gerektirirler.



Parola Politikaları (Zenginleştirme)

API'ye kayıt olan kullanıcıların güçlü parolalar kullanmasını sağlamak gerekir.

****Politikalar:****

1. Minimum uzunluk (örn: 12 karakter).
2. Karakter çeşitliliği (büyük/küçük harf, rakam, sembol).
3. Yaygın olarak kullanılan (leak edilmiş) parolaların reddedilmesi (Have I Been Pwned API entegrasyonu).

Çıkarım: Başarılı Güvenlik Entegrasyonu

Bu modülün sonunda, acil güvenlik fiyaskosu başarıyla giderilmiştir [2].

Artık IoC verileri yetkisiz erişime açık değildir. Tüm kritik endpoint'ler, güvenli parola hashing, OAuth2 standardına uygun token akışı ve JWT imzası ile korunmaktadır [3, 7, 9].

Geliştirilen yapı, güvenli API geliştirmenin modern temellerini oluşturmaktadır.

Tekrarlama: JWT İmza Değeri

İmza (Signature), token'i oluşturan üç parçanın (Header, Payload, Gizli Anahtar) kriptografik özetidir.

Eğer bir saldırgan Payload'daki bir karakteri bile değiştirirse, sunucu imza doğrulamasını yaptığında yeni hesaplanan imza, mevcut imza ile eşleşmez. Sunucu, bu durumda token'i anında reddeder.



Uygulama Alanları: Farklı Flow'lar

JWT ve OAuth2 sadece API güvenliğinde değil, çeşitli uygulamalarda kullanılır:

1. ****SSO (Single Sign-On):**** Kullanıcı bir uygulamaya girdiğinde, diğer tüm uygulamalara otomatik olarak erişim sağlar.
2. ****Mobil Uygulamalar:**** Mobil istemcilerde kimlik doğrulama için genellikle OAuth2 flow'ları kullanılır.
3. ****Cihaz Doğrulama:**** IoT cihazları bile kimlik doğrulaması için basitleştirilmiş token mekanizmalarını kullanabilir.

Token Ömrü ve Güvenlik Riski İlişkisi

Daha kısa token ömrü, çalınan token riskini azaltır ancak sunucunun sürekli token yenileme isteği alması yükü artırır.

Bu modüldeki örnekte, erişim token'ının kısa (örn. 15 dakika) ayarlanması tipik bir başlangıç güvenlik seviyesidir. Daha uzun oturumlar için Refresh Token mekanizması şarttır [4].



API Ağ Geçidi Entegrasyon Örneği

Büyük ölçekli sistemlerde, token doğrulama mantığı ('auth.get_current_user' benzeri işlevler) genellikle bir ara yazılım (middleware) veya ağ geçidi tarafından yapılır.

Bu, FastAPI uygulamamızın sadece token'ın geçerli olduğuna güvenerek iş mantığını yürütmesine olanak tanır. Böylece API, kimlik doğrulama yükünden kurtulur.



Hashing Maliyetinin Ayarlanması

Bcrypt, 'cost factor' (maliyet faktörü) adı verilen bir parametre ile ne kadar yavaş çalışacağını belirler.

****Avantajı:**** Donanım geliştikçe (daha hızlı işlemciler/GPU'lar), cost factor artırılarak hash'lerin kırılması zorlaştırılabilir. Bu, uzun vadeli bir güvenlik stratejisidir.

Passlib kütüphanesi, bu maliyet ayarını kolayca yapmamızı sağlar.

Yetkilendirme Kontrolü (Role Check)

'auth.get_current_user' fonksiyonu tarafından döndürülen 'current_user' objesini kullanarak yetkilendirme (authorization) kontrolü yapılabilir:

Örn: 'if current_user.role != 'admin': raise HTTPException(status_code=403, detail=Permission denied)'

Bu kontrol, IoC verilerini silebilecek kullanıcıları hassas bir şekilde sınırlandırır.



JSON Web Key (JWK) Setleri

Asimetrik imza (RS256) kullanıldığında, API'nin genel anahtarını diğer servislerle paylaşması gerekir.

JWKS (JSON Web Key Set), bu genel anahtarları standart bir JSON formatında yayımlamak için kullanılır. Bu sayede, token'i tüketen servisler, genel anahtarı kolayca çekip JWT'nin imzasını doğrulayabilir.



Özet: Ders Akışı Zaman Çizelgesi

Ders akışı, adım adım güvenliğin inşa edilmesini hedeflemiştir [4, 5, 10]:

1. Kurulum (15 dk) [5]
2. Model/Şema Tanımları (30 dk) [5]
3. auth.py / Güvenlik Mantığı (90 dk) [4]
4. main.py / Endpoint Entegrasyonu (45 dk) [10]
5. Test ve Tartışma (Kilit düğmesi) [9]

Parola Sıfırlama Mekanizması

Güvenlik modülünün önemli bir parçası, kullanıcıların parolalarını güvenli bir şekilde sıfırlama yeteneğidir.

****Akış:****

1. Kullanıcı e-posta ile sıfırlama talebi gönderir.
2. Sunucu, tek kullanımlık, kısa ömürlü ve kriptografik olarak güvenli bir token oluşturur ve bu token'i içeren bir bağlantıyı e-posta ile gönderir.
3. Kullanıcı, bu bağlantıyı kullanarak yeni bir parola belirler. Yeni parola, yine bcrypt ile hash'lenir.



Token Doğrulama (Validate Token) Mimarisi

Token doğrulama sürecinde, 'python-jose' kütüphanesinin 'jwt.decode' fonksiyonu kullanılır. Bu fonksiyon, anahtar (SECRET_KEY) ve algoritma parametrelerini alarak imza kontrolünü otomatik olarak yapar.

Eğer imza eşleşmezse veya token'ın süresi dolmuşsa, kütüphane otomatik olarak bir istisna (exception) fırlatır ve bu da 'get_current_user' tarafından 401 hatasına çevrilir [6].



Çift Faktörlü Kimlik Doğrulama (MFA)

MFA, kullanıcı adı ve parolaya ek olarak ikinci bir doğrulama faktörünün (örn: telefona gelen kod, biyometrik veri) kullanılmasını zorunlu kılar.

FastAPI uygulamasında, başarılı JWT oluşturulmadan önce kullanıcıdan bu ikinci faktörün doğrulanması istenebilir. Bu, çalınan parolaların bile kullanılamamasını sağlar.



Bu modülde öğrendiğimiz tüm konular, aşağıdaki temel standartlara dayanmaktadır:

1. ****RFC 7519:**** JSON Web Token (JWT) standardı.
2. ****RFC 6749:**** The OAuth 2.0 Authorization Framework.
3. ****OWASP Top 10:**** API güvenliği için en güncel tehdit listesi.

Öğrencilerin bu standartları detaylıca incelemesi tavsiye edilir.

Güvenlik Kapanış: API'niz Artık Korumalı

IoC verilerinin korunması görevi, başarılı bir şekilde tamamlanmıştır. Uygulama, güçlü kimlik doğrulama, yetkilendirme ve güvenli parola yönetimi ilkelerine uygun olarak tasarlanmıştır [1, 3].

Bu modülde edinilen bilgiler, herhangi bir modern siber güvenlik projesinde temel alınması gereken pratik becerilerdir.



API Güvenliđi Neden Hayati Önem Taşıır?

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

